

Module 1: Debugging AttackMate Playbooks and Lab Environments

This is a reference for diagnosing problems when attack chains fail. It covers the tools and commands you need to verify every layer: network reachability, open ports, running services, routing, and firewall rules.

AttackMate Itself

Enabling Debug Output

```
# Shows variable dumps, resolved values, and full execution trace
attackmate --debug playbook.yml
```

Inspecting Variables Mid-Playbook

Insert a **debug** command anywhere to print the current state of a variable:

```
- type: debug
  cmd: "TARGET is: $TARGET"

- type: debug
  cmd: "Last result: $RESULT_STDOUT"

- type: debug
  cmd: "Return code: $RESULT_RETURNCODE"
```

Checking **only_if** Logic

only_if silently skips commands. If a step is not running, print what the condition sees:

```
- type: debug
  cmd: "PORT value is: '$PORT'"

- type: shell
  cmd: curl http://$TARGET:$PORT
  only_if: $PORT == 80
```

Variables are always saved as strings in the variable store. **\$PORT == 80** compares "80" to "80", which works. But **\$PORT == 080** will fail silently.

Network Connectivity

Ping: Basic Reachability

ping sends ICMP Echo Request packets to a host and waits for ICMP Echo Reply packets in return. It measures whether the host responds at all and how long the round trip takes (latency). Ping operates at the network layer (Layer 3) and does not test any specific service or port, it only tells you whether the machine is reachable at the IP level.

```
# Send 4 packets to check if the host is up
ping -c 4 <TARGET_IP>

# Faster: 1 packet, 1-second timeout
ping -c 1 -W 1 <TARGET_IP>

# Check if a ping succeeds in a playbook step
```

```
- type: shell
  cmd: ping -c 1 -W 2 $TARGET
  error_if: $RESULT_RETURNCODE != 0
```

Note: Some hosts block ICMP. A failed ping does not mean the host is down. Follow up with a port scan.

Checking if a Specific Port is Open

A **port** is a numbered endpoint (0–65535) that an operating system uses to route incoming network traffic to the right process. When a service is running and waiting for connections on a port, that port is **open**, meaning a client can connect to it. A closed port means nothing is listening there, and the OS will immediately refuse the connection. An open port does not tell you that the service is working correctly; it only tells you that something accepted the TCP handshake.

```
# nc (netcat): attempt TCP connection, timeout after 3 seconds
nc -zv <TARGET_IP> <PORT>
nc -zvw3 <TARGET_IP> 22

# Check multiple ports
nc -zvw3 <TARGET_IP> 21 22 80 443 4444

# Using /dev/tcp (no nc required, works in bash)
timeout 3 bash -c "echo > /dev/tcp/<TARGET_IP>/<PORT>" && echo "open" ||
echo "closed"
```

Scan for Open Ports

```
# Fast scan of the most common 1000 ports
nmap <TARGET_IP>

# Scan specific ports
nmap -p 21,22,80,443,4444,55553 <TARGET_IP>

# Service version detection on open ports
nmap -sV -p 21,22,80 <TARGET_IP>

# Scan all 65535 ports (slow, but thorough)
nmap -p- <TARGET_IP>
```

Test UDP Ports

UDP (User Datagram Protocol) is a connectionless transport protocol. Unlike TCP, UDP does not perform a handshake before sending data and does not guarantee delivery or ordering. Services like DNS (port 53), SNMP (port 161), and TFTP use UDP because they prioritize speed over reliability. Testing UDP ports is harder than TCP: a lack of response can mean the port is open (the service received the packet and sent no reply), filtered (a firewall dropped it), or closed (the OS sent an ICMP "port unreachable" error). For this reason, UDP scanning is slower and less reliable.

```
# UDP scan (requires root)
sudo nmap -sU -p 53,161 <TARGET_IP>

# nc UDP mode
nc -zuv <TARGET_IP> 53
```

Netcat (nc) as a Debugging Tool

Netcat (**nc**) is a command-line tool that reads and writes raw data over TCP or UDP connections. It can act as either a client (connecting to a remote port) or a server (listening for incoming connections). Unlike **curl** or **ssh**, netcat sends and receives raw bytes without any protocol framing, making it ideal for testing whether a port is reachable, grabbing service banners, or simulating a simple listener for reverse shells.

Manually Trigger a Service

```
# Connect to FTP and read the banner
nc <TARGET_IP> 21

# Connect to HTTP and send a raw request
echo -e "GET / HTTP/1.0\r\n\r\n" | nc <TARGET_IP> 80

# Connect to a raw shell or backdoor port
nc <TARGET_IP> 6200
```

Listen for Incoming Connections

Use this to verify that a reverse shell or payload is actually calling back:

```
# Listen on port 4444, print whatever arrives
# -l listen mode (wait for an incoming connection instead of connecting
out)
# -v verbose (print status messages like "listening on..." and "connection
from...")
# -n no DNS resolution (use raw IP addresses, faster and avoids lookup
failures)
# -p specify the port to listen on
nc -lvnp 4444
```

Then run your payload on the target and watch for the connection.

Routing and Network Path

Show the Routing Table

The **routing table** is a list of rules that the kernel uses to decide where to send outgoing packets. Each entry specifies a destination network, the network interface to use, and optionally a gateway (next-hop router). When you send a packet to a target IP, the kernel looks up the most specific matching route and forwards the packet accordingly. If no route exists for the destination, the packet is dropped. The default route (**default** or **0.0.0.0/0**) is the fallback used for any IP that does not match a more specific entry.

```
ip route show
```

A typical routing table looks like this:

```
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel src 192.168.1.50
10.0.0.0/8 dev eth1 proto kernel src 10.0.0.5
```

- **default via 192.168.1.1 dev eth0**: all traffic not matching a specific route goes to the gateway **192.168.1.1** out of **eth0**
- **192.168.1.0/24 dev eth0**: traffic to any address in **192.168.1.*** goes directly out **eth0** (same LAN)
- **10.0.0.0/8 dev eth1**: the **10.*.*.*** network is reachable directly on **eth1**

Look for: does a route exist to **<TARGET_IP>**? What interface is used? What is the gateway?

Trace the Network Path

traceroute reveals the sequence of routers (hops) a packet passes through on its way to the destination. It works by sending packets with incrementally increasing TTL (Time To Live) values. Each router that forwards a packet decrements the TTL by one. When TTL reaches zero, the router discards the packet and sends back an ICMP "time exceeded" message, revealing its IP address. By repeating this with TTL = 1, 2, 3, ... traceroute maps the full path. If a hop shows * * *, that router is either dropping probe packets or not sending ICMP replies (common on firewalls and cloud infrastructure).

```
# Show each hop between attacker and target
traceroute <TARGET_IP>

# Using ICMP (may work where UDP traceroute is blocked)
traceroute -I <TARGET_IP>

# Using TCP on a specific port (good through firewalls)
traceroute -T -p 80 <TARGET_IP>
```

Show Network Interfaces and IPs

A **network interface** is the software representation of a network connection, either a physical NIC (network interface card) or a virtual one created by the OS. Each interface has a name (e.g., **eth0**, **ens3**, **lo**) and one or more IP addresses assigned to it. The loopback interface (**lo**) is always present and handles traffic to **127.0.0.1** without going to the network. In lab environments you typically have at least one interface for the management network and one for the lab/attack network. **When setting LHOST in a payload, you need the IP of the interface that the target can actually reach.**

```
# All interfaces and their addresses
ip addr show

# Just the IP on a specific interface
ip addr show eth0
```

Check Active Connections

```
# All established TCP connections
ss -tn state established

# All listening sockets (what is this machine serving?)
# -t show TCP sockets only
# -l show only listening sockets (services waiting for connections)
# -n show numeric addresses and port numbers (no DNS/service name resolution)
# -p show the process name and PID that owns each socket
ss -tlnp

# Watch for connections appearing on a specific port
watch -n 1 "ss -tn | grep :4444"
```

Firewall Rules

iptables is the Linux kernel's built-in packet filtering system. It organizes rules into **chains** (ordered lists) within **tables**:

- The **filter** table (the default) has three chains: **INPUT** (incoming packets destined for this host), **OUTPUT** (packets originating from this host), and **FORWARD** (packets being routed through this host).
- The **nat** table handles address translation and port forwarding.

Each chain has a **default policy** (usually **ACCEPT** or **DROP**) that applies when no rule matches. Rules are checked top to bottom; the first match wins.

What a rule looks like:

```
Chain INPUT (policy ACCEPT)
num  target  prot  opt  source        destination
1    ACCEPT  tcp   --   0.0.0.0/0      0.0.0.0/0      tcp dpt:22
2    DROP    tcp   --   10.0.0.5       0.0.0.0/0
3    ACCEPT  all   --   0.0.0.0/0      0.0.0.0/0      state
RELATED, ESTABLISHED
```

- **target**: what to do with a matching packet (**ACCEPT** lets it through, **DROP** silently discards it, **REJECT** discards it and sends an error back)
- **prot**: protocol (**tcp**, **udp**, **all**)
- **source** / **destination**: match by IP or subnet; **0.0.0.0/0** means any address
- **dpt**: destination port

To diagnose a connectivity problem, read the INPUT chain from top to bottom and ask: does a rule match my traffic before reaching the default policy?

Inspect Current Rules

```
# Show iptables rules with line numbers
sudo iptables -L -n -v --line-numbers

# Show only the INPUT chain (incoming traffic rules)
sudo iptables -L INPUT -n -v --line-numbers

# Show the OUTPUT chain
sudo iptables -L OUTPUT -n -v --line-numbers

# Show NAT table (for port forwarding / masquerade)
sudo iptables -t nat -L -n -v
```

Allow Incoming Traffic from a Specific IP and Port

```
# Allow TCP from a specific source IP to a specific destination port
sudo iptables -A INPUT -s <SOURCE_IP> -p tcp --dport <PORT> -j ACCEPT

# Example: allow the target to call back on port 4444
sudo iptables -A INPUT -s 192.168.1.100 -p tcp --dport 4444 -j ACCEPT

# Allow from any source (useful for testing, tighten up later)
sudo iptables -A INPUT -p tcp --dport 4444 -j ACCEPT
```

Block Traffic

```
# Block all inbound traffic from an IP
sudo iptables -A INPUT -s <IP> -j DROP

# Block a specific port
sudo iptables -A INPUT -p tcp --dport <PORT> -j DROP
```

Remove a Rule

```
# List rules with line numbers first, then delete by number
sudo iptables -L INPUT -n --line-numbers
sudo iptables -D INPUT <LINE_NUMBER>

# Or delete by repeating the rule with -D instead of -A
sudo iptables -D INPUT -p tcp --dport 4444 -j ACCEPT
```

UFW (if installed)

```
# Show current rules
sudo ufw status verbose

# Allow a port from any IP
sudo ufw allow 4444/tcp

# Allow from a specific IP
sudo ufw allow from 192.168.1.100 to any port 4444

# Delete a rule
sudo ufw delete allow 4444/tcp
```

Checking if Metasploit is Running

Check msfrpcd

```
# Is msfrpcd listening on port 55553?
ss -tlnp | grep 55553

# More detailed: show the process holding the port
ss -tlnp sport = :55553
```

Check if the msfrpcd Process is Running

```
# Find it by name
ps aux | grep msfrpcd

# Alternative
pgrep -a msfrpcd
```

Start msfrpcd if it is Not Running

```
# Bind to localhost, password "msf", SSL enabled (default)
msfrpcd -P msf -a 127.0.0.1

# Verify it started
sleep 2 && ss -tlnp | grep 55553
```

Verify msfrpcd is Responding (in a Playbook)

```
- type: shell
  cmd: ss -tlnp | grep 55553
  error_if: $RESULT_STDOUT == ""
```

Access msfconsole Manually

```
msfconsole

# Once inside, list active sessions
msf6 > sessions -l

# Kill all sessions
msf6 > sessions -K

# Check what listeners are running
msf6 > jobs -l
```


Checking if Sliver is Running

Check the Sliver Server Process

```
# Is sliver-server running?
ps aux | grep sliver-server
pgrep -a sliver

# Check what ports it is listening on (default: 31337 for operators, 443/80
for implants)
ss -tlnp | grep -E "31337|443|80"
```

Start the Sliver Server

```
# Run in the foreground (for debugging)
sudo sliver-server

# Run as a background daemon (if systemd service is configured)
sudo systemctl start sliver

# Check service status
sudo systemctl status sliver
```

Connect with the Sliver Client

```
# Connect using your operator config
sliver-client --config ~/.sliver-client/configs/operator.cfg

# Once inside the Sliver console, list active sessions
sliver > sessions

# List active beacons
sliver > beacons

# List running jobs (listeners)
sliver > jobs
```

Test Operator gRPC Port Connectivity

```
# Default operator port is 31337
nc -zvw3 127.0.0.1 31337
```

Check Implant Listener Ports

```
# If Sliver is using HTTPS on 443 or HTTP on 80 for implants  
ss -tlnp | grep -E ":443|:80"
```

SSH: Manual Testing and Troubleshooting

Connect Manually

```
# Standard connection  
ssh user@<TARGET_IP>  
  
# Specify a key  
ssh -i /path/to/key.pem user@<TARGET_IP>  
  
# Use a non-standard port  
ssh -p 2222 user@<TARGET_IP>  
  
# Verbose output (shows exactly why a connection fails)  
ssh -vvv user@<TARGET_IP>
```

Legacy Key Algorithm Support (Metasploitable2)

Older SSH servers use deprecated key algorithms that modern OpenSSH rejects. Add to `~/.ssh/config`:

```
Host <TARGET_IP>  
  HostKeyAlgorithms +ssh-rsa,ssh-dss  
  PubkeyAcceptedKeyTypes +ssh-rsa,ssh-dss
```

Or pass on the command line:

```
ssh -o HostKeyAlgorithms=+ssh-rsa,ssh-dss \  
  -o PubkeyAcceptedKeyTypes=+ssh-rsa,ssh-dss \  
  user@<TARGET_IP>
```

Run a Single Remote Command

```
ssh user@<TARGET_IP> "id && hostname && ip addr"
```

Check SSH Key Permissions

SSH refuses keys with overly permissive permissions:

```
chmod 600 /path/to/key.pem
chmod 700 ~/.ssh
```

How Linux file permissions work: Every file has three sets of permissions — for the **owner**, the **group**, and **everyone else** (world). Each set has three bits: **r** (read = 4), **w** (write = 2), **x** (execute = 1). The three-digit number in **chmod** is the sum of those bits for owner, group, and world respectively:

chmod value	Meaning
600	owner: read+write (6), group: none (0), world: none (0)
644	owner: read+write (6), group: read (4), world: read (4)
700	owner: read+write+execute (7), group: none, world: none
755	owner: all (7), group: read+execute (5), world: read+execute (5)

You can inspect current permissions with **ls -l**:

```
-rw----- 1 user user 1679 Apr  1 10:00 key.pem
```

The first 10 characters: **-** (file type), then **rw-----** (permissions: owner has read+write, group and world have nothing).

SSH enforces that private keys are readable only by their owner (**600**). If the key file is readable by others, SSH treats it as compromised and refuses to use it.

Test SSH Port is Actually Reachable

```
nc -zvw3 <TARGET_IP> 22

# Grab the SSH banner manually
nc <TARGET_IP> 22
```

HTTP: Checking Web Services

Fetch a URL and See the Response

```
# Follow redirects, show response headers
curl -Lv http://<TARGET_IP>/

# Ignore TLS errors (self-signed certs)
curl -k https://<TARGET_IP>/

# Show only the HTTP status code
```

```
curl -s -o /dev/null -w "%{http_code}" http://<TARGET_IP>/

# POST data to a form
curl -X POST -d "username=admin&password=admin" http://<TARGET_IP>/login
```

Check if a Web Server is Listening

```
# nc: send minimal HTTP request
echo -e "GET / HTTP/1.0\r\n\r\n" | nc -w3 <TARGET_IP> 80
```

Test HTTP in a Playbook

```
- type: shell
  cmd: curl -s -o /dev/null -w "%{http_code}" http://$TARGET
  error_if: $RESULT_STDOUT != 200
```

Diagnosing Reverse Shell Callbacks

When a payload should call back but doesn't, work through each layer.

Step 1: Verify the Listener is Running

```
# Is something listening on your callback port?
ss -tlnp | grep :4444
```

If nothing is listening, the payload has nowhere to connect to. Start your listener first.

Step 2: Test Reachability from the Target's Perspective

From the target (if you have any access):

```
# Can the target reach the attacker on that port?
nc -zvw3 <ATTACKER_IP> 4444
```

From the attacker side, use a `nc` listener to catch a test connection:

```
nc -lvnp 4444
# On target:
nc <ATTACKER_IP> 4444
```

Step 3: Check Routing from Target to Attacker

```
# On the target (if accessible)
ip route show
traceroute <ATTACKER_IP>
```

Step 4: Check Firewall on the Attacker

```
# Is the attacker's firewall blocking inbound on the callback port?
sudo iptables -L INPUT -n -v | grep 4444

# Open it
sudo iptables -A INPUT -p tcp --dport 4444 -j ACCEPT
```

Step 5: Check LHOST is Correct

A common mistake is setting **LHOST** to **127.0.0.1** or to the wrong interface. The payload uses **LHOST** to know where to call back. It must be reachable from the target:

```
# What IP does the target use to reach the attacker?
ip route get <TARGET_IP>
```

Use the IP shown under **src** as your **LHOST**.

Process and System Checks

What is Listening on a Port?

```
# Which process is bound to port 4444?
ss -tlnp sport = :4444

# Alternative
sudo lsof -i :4444
```

Kill a Process by Port

```
# Find the PID
ss -tlnp sport = :4444

# Kill it
sudo kill <PID>
```

Check System Logs for Errors

`journalctl` queries the **systemd journal**, the centralized log store on modern Linux systems. All messages from the kernel, system services, and any process managed by systemd end up here. It replaces the older practice of reading scattered files in `/var/log/`. The flags used most often for debugging:

- `-x`: add explanatory text (catalog entries) for some error messages
- `-e`: jump to the end of the log (most recent entries)
- `-f`: follow (tail) new messages as they arrive
- `--since "5 minutes ago"`: limit output to a recent time window
- `-u <service>`: show logs for a specific unit, e.g., `-u sliver` or `-u ssh`

```
# Recent kernel and system messages
journalctl -xe --since "5 minutes ago"

# Tail a specific log file
tail -f /var/log/syslog
tail -f /var/log/auth.log
```

Memory and CPU (when a tool is hanging)

`ps aux` lists all currently running processes. The flags:

- `a`: show processes from all users (not just yours)
- `u`: show user-oriented output (username, CPU%, memory%, start time)
- `x`: include processes not attached to a terminal (background daemons)

The key columns in the output:

Column	Meaning
USER	owner of the process
PID	process ID (use this to kill the process)
%CPU	CPU usage percentage
%MEM	percentage of physical RAM in use
STAT	process state: <code>S</code> = sleeping, <code>R</code> = running, <code>Z</code> = zombie, <code>D</code> = uninterruptible wait
COMMAND	the executable and its arguments

```
# Interactive process list
htop

# One-shot snapshot
ps aux --sort=-%cpu | head -20
```

Quick Reference Table

Problem	First Command to Run
Is the target reachable?	<code>ping -c 1 <TARGET_IP></code>
Is a specific port open?	<code>nc -zvw3 <TARGET_IP> <PORT></code>
What ports are open?	<code>nmap -p- <TARGET_IP></code>
Is msfrpcd running?	<code>ss -tlnp grep 55553</code>
Is Sliver server running?	<code>ps aux grep sliver-server</code>
Is my listener up?	<code>ss -tlnp grep :<PORT></code>
Is firewall blocking input?	<code>sudo iptables -L INPUT -n -v</code>
Allow a callback port	<code>sudo iptables -A INPUT -p tcp --dport <PORT> -j ACCEPT</code>
What is my attacker IP?	<code>ip addr show</code>
What IP to use as LHOST?	<code>ip route get <TARGET_IP></code>
Why is SSH failing?	<code>ssh -vvv user@<TARGET_IP></code>
Why is a step being skipped?	Insert <code>debug</code> before it, print the condition variable
Is AttackMate substituting correctly?	<code>attackmate --debug playbook.yml</code>
Did the payload reach the right port?	<code>nc -lvnp <PORT></code> on attacker, run payload